Francisco Redondo, ist1106117
Vasco Pereira, ist1103368
Group 16

Homework 4: Model-Free RL Temporal Difference and Exploration

1. SARSA vs Q-Learning

1.1 Q-Learning uses the greedy action of the next state to update its values. Because we are working with costs, we will want to minimize Q-value for the next state.

The standard cost-based Q-learning update formula is:

$$Q(S_t, a_t) \leftarrow Q(S_t, a_t) + \alpha \left[c_t + \gamma \min_{a'} Q(S_{t+1}, a') - Q(S_t, a_t) \right]$$

Plugging in the specific values from your third transition where $S_t = S_H$, $a_t = a_S$, $c_t = 0$, and $S_{t+1} = S_H$:

$$Q(S_H, a_S) \leftarrow Q(S_H, a_S) + \alpha \left[0 + \gamma \min_{a'} Q(S_H, a') - Q(S_H, a_S) \right]$$

And this is the final update rule for the third transition.

1.2 Q-Learning is an *off-policy* algorithm. It computes the Temporal Difference (TD) target using the absolute best (greedy) action for the next state, regardless of what the agent actually does. Hence, it will evaluate $\min_{a'} Q(S_H, a')$. Conversely, SARSA is an *on-policy* algorithm. It updates its values based on the action the agent *actually* takes. Since the agent chose a_B for the fourth transition, SARSA will use $Q(S_H, a_B)$ for its TD target.

So they would only be identical if a_B coincidentally happens to be the greedy action (i.e., if $Q(S_H, a_B) \leq Q(S_H, a_S)$).

1.3 Yes, both Q-Learning and SARSA use *bootstrapping*. *Bootstrapping* in reinforcement learning means updating a value estimate using other current value estimates (in this case, the Q-values of the next state, S_H), rather than waiting until the end of an episode to see the final, true accumulated cost.

2. Exploration-Exploitation

2.1 Since the environment uses costs, the agent prefers the *lowest* value. Therefore the greedy action is a_S (because 5 < 15) and the non-greedy action is a_B.

In an ϵ-greedy policy with 2 possible actions ($|A|$) - a_S and a_B - the agent exploits the greedy action with a probability of $1-\epsilon$, and explores by choosing random with a probability of ϵ.
That means that the probability of choosing action a_B is simply:

$$P(a_B) = \frac{\epsilon}{|A|} = \frac{0.2}{2} = 0.1$$

2.2 The main risk is premature convergence to a sub-optimal policy. If the agent stops exploring too early, it will greedily exploit only the actions it currently thinks are best. It completely loses the chance to discover other actions or states that might ultimately yield much lower costs and consequently better results.

3. Model-Based Learning (MLE)

3.1 To calculate the MLE for the transition probabilities, we need to count how many times a specific transition occurred and divide it by the total number of times that specific state-action pair was visited.

Based on the environment description, the agent experiences this exact trajectory:

$$(S_L, a_S, 10, S_L) \rightarrow (S_L, a_B, 20, S_H) \rightarrow (S_H, a_S, 0, S_H)$$

Let's isolate the transitions where the agent was in state S_L and took action a_S:

- This specific state-action pair (S_L, a_S) only happens once in the entire trajectory, which is the very first step.
- During that single occurrence, the agent transitions to state S_L.

Therefore, the MLE probabilities are:

- $P(S_L|S_L, a_S) = \frac{1}{1} = 1$
- $P(S_H|S_L, a_S)' = \frac{0}{1} = 0$

Initial State (s)	Action (a)	Next State (s′)	Transition Count
S_L	a_S	S_L	1
S_L	a_S	S_H	0

Fig 1. Transition Table Count

4. Policy Improvement Step

4.1 For a greedy policy, the agent will deterministically choose the action that has the highest Q-value for each given state.
Here are the Q-values the agent has learned:

- $Q(S_L, a_S) = 40$
- $Q(S_L, a_B) = 30$
- $Q(S_H, a_S) = 10$
- $Q(S_H, a_B) = 5$

To find the new deterministic policy $\pi(s)$, we just compare the actions for each state:

- **For state S_L:** We compare $Q(S_L, a_S) = 40$ against $Q(S_L, a_B) = 30$. Since 40 is greater than 30, the greedy action is a_S.
  - $\pi(S_L) = a_S$
- **For state S_H:** We compare $Q(S_H, a_S) = 10$ against $Q(S_H, a_B) = 5$. Since 10 is greater than 5, the greedy action is a_S.
  - $\pi(S_H) = a_S$

4.2 A Softmax policy converts Q-values into a probability distribution, meaning actions with higher Q-values have a higher probability of being chosen, but other actions still have a non-zero chance. The formula for the probability of choosing action a in state s with temperature τ is:

$$P(a|s) = \frac{e^{Q(s,a)/\tau}}{\sum_{a'} e^{Q(s,a')/\tau}}$$

Step-by-step calculation:

1. **Identify the relevant Q-values:** $Q(S_L, a_S) = 40$ and $Q(S_L, a_B) = 30$.

2. **Calculate the exponent for action a_S (the numerator):** $e^{40/10} = e^4$
3. **Calculate the exponent for action a_B:** $e^{30/10} = e^3$
4. **Calculate the sum of all exponents for state S_L (the denominator):** $e^4 + e^3$

Putting it all together, the probability calculation is:

$$P(a_S|S_L) = \frac{e^4}{e^4 + e^3}$$

$$P(a_S|S_L) \approx 0.731$$

So, the probability of the agent choosing action a_S in state S_L under this Softmax policy is approximately **73.1%**.